

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE
DELL'INFORMAZIONE



CORSO DI LAUREA IN INFORMATICA

INSEGNAMENTO DI LABORATORIO DI SISTEMI OPERATIVI

ANNO ACCADEMICO 2025/2026

Progettazione di una architettura client-server per Forza 4

Autore:

Castaldo Carmine

Matricola: N86005226

Docente:

Prof.ssa Alessandra Rossi

Indice

Introduzione e Analisi dei Requisiti	2
1 ARCHITETTURA DI RETE E PROTOCOLLO APPLICATIVO	3
1.1 Protocollo di Trasporto	3
1.2 Design del Protocollo Custom a Livello Applicativo	3
2 PROGETTAZIONE DEL SERVER CONCORRENTE IN C	5
2.1 Parallelismo	5
2.2 Gestione Dinamica delle Risorse	5
2.3 Sincronizzazione e Mutua Esclusione tramite Mutex	6
2.4 Gestione delle Disconnessioni	6
2.5 Politiche di Gestione delle Stanze e Stati della Partita	6
3 PROGETTAZIONE DEL CLIENT IN PYTHON	8
3.1 User Interface e Threading asincrono	8
3.2 Sincronizzazione Thread-Safe	8
4 ISOLAMENTO E ORCHESTRAZIONE DEL DEPLOYMENT	9
4.1 Containerizzazione dell'Infrastruttura: Server e Client	9
4.2 Integrazione e Networking tramite Docker Compose	9

Introduzione e Analisi dei Requisiti

La presente relazione tecnica si prefigge lo scopo di documentare e motivare analiticamente le decisioni progettuali che hanno guidato lo sviluppo dell'intera architettura client-server. Per facilitare la comprensione delle dinamiche del sistema, l'esposizione seguirà il naturale flusso logico degli eventi di rete: dall'inizializzazione della sessione utente alla gestione delle stanze, fino ad arrivare alle fasi di gioco attivo, contesto in cui divengono cruciali i meccanismi di sincronizzazione concorrente. Questa struttura espositiva è stata concepita per contestualizzare ogni singola scelta tecnologica e implementativa rispetto alla precisa fase evolutiva in cui si trova l'applicativo.

Al fine di fornire un chiaro quadro di riferimento, si richiama di seguito la specifica originale del progetto assegnato:

Progetto 1: Forza 4

Lo studente dovrà realizzare un server multi-client per giocare a Forza 4. Il gioco è pensato per due partecipanti. Ogni giocatore, una volta collegato, deve poter creare una o più partite, ma deve poter giocare attivamente solo a una partita alla volta.

Quando un giocatore crea una nuova partita, questa viene inizialmente registrata dal server e resa disponibile agli altri utenti. Ogni partita deve essere identificata in maniera univoca, così che i client possano richiedere l'accesso indicando il relativo identificativo.

Una volta creata una partita, il suo stato iniziale può essere considerato come una nuova creazione, in cui viene semplicemente registrata. Successivamente, la partita entra nello stato di attesa, durante il quale gli altri giocatori collegati al server vengono informati della sua presenza e possono chiedere di partecipare.

Quando un nuovo giocatore richiede di unirsi alla partita, il creatore può accettare o rifiutare la richiesta. Solo nel caso venga accettata, la partita passa nello stato di gioco in corso e inizia effettivamente la partita di Forza 4.

Durante la partita, i due giocatori vengono informati sullo stato della griglia e su chi ha il turno, mentre gli altri client collegati al server ricevono messaggi diversi e vengono solo messi a conoscenza che quella partita è in corso e non è più possibile parteciparvi.

Alla conclusione della partita, lo stato diventa terminata. A questo punto il server deve inviare ai due giocatori coinvolti un esito personalizzato, distinguendo tra vittoria, sconfitta o pareggio, a seconda del risultato ottenuto. Gli altri client collegati riceveranno soltanto la notifica che la partita si è conclusa.

Una volta terminata, i giocatori possono decidere se iniziare una nuova partita. In tal caso la griglia viene ripristinata e il gioco può ricominciare.

Capitolo 1

ARCHITETTURA DI RETE E PROTOCOLLO APPLICATIVO

1.1 Protocollo di Trasporto

Il codice dell'infrastruttura di rete è suddiviso in due componenti principali, uno per il lato server e uno per il lato client, identificati rispettivamente nei moduli `connectionController` e `network_controller`.

Il `connectionController` consiste in un modulo sviluppato in C che racchiude l'intera logica di gestione della rete lato backend. I compiti principali di questo componente sono:

- **Gestione dei client:** Tracciare costantemente lo stato e l'identità di tutti i client attivi. Gestire l'apertura e la chiusura sicura delle connessioni socket. Generare un thread dedicato (client-specific) per ogni nuova connessione, occupandosi del rilascio controllato del thread stesso e delle sue risorse non appena la connessione viene terminata.
- **Gestione dello scambio di messaggi:** Gestire la ricezione dei messaggi provenienti dai client e l'invio delle risposte o delle notifiche da parte del server sulla rete.

Il `network_controller` è invece una classe scritta in Python che astrae l'intera logica di comunicazione di rete lato client, esponendo un'interfaccia semplificata di livello superiore verso i componenti grafici dell'applicazione.

Il protocollo di rete a cui si fa affidamento è il TCP (Transmission Control Protocol). La scelta di TCP è critica poiché Forza 4 si basa sulla sincronizzazione rigida dello stato della griglia. Se anche un singolo messaggio contenente una mossa venisse corrotto, perso o invertito, i client si de-sincronizzerebbero istantaneamente, compromettendo irrimediabilmente l'intera partita. TCP elimina questo rischio garantendo l'integrità assoluta e la sequenzialità di ogni singolo byte trasmesso. Si è evitata la scelta del protocollo UDP, quindi, per motivi di affidabilità dell'applicazione.

1.2 Design del Protocollo Custom a Livello Applicativo

Per lo scambio di messaggi tra client, server e moduli interni viene utilizzato un sistema a base di stringhe, preferibile ad un sistema a base dei singoli bit che pur avendo il vantaggio della leggerezza, rende i messaggi più soggetti ad errori e complica il debugging.

Sono definiti due tipi di messaggi principali:

Parametrici: Dopo l'azione è presente un parametro, che viene interpretato in base al comando:

- **MOVE %d:** Inviato dal client durante il proprio turno di gioco. L'intero rappresenta l'indice della colonna (da 0 a 6) all'interno della quale il giocatore desidera inserire il proprio disco.
- **JOIN %d:** Inviato da un client nella lobby principale. L'intero rappresenta l'identificativo univoco (`id_match`) della partita (o stanza) a cui l'utente intende aggregarsi per sfidare l'host.

Non Parametrici:

- **CREATE:** Richiesta inviata da un client per istanziare una nuova partita nella lobby, assumendo il ruolo di host in attesa di uno sfidante.
- **ACCEPT:** Inviato dall'host per accettare la richiesta di sfida ricevuta da un altro utente ed avviare ufficialmente la partita.
- **REJECT:** Inviato dall'host per rifiutare la richiesta di ingresso nella stanza da parte di un utente, rimettendo la stanza in stato di attesa.
- **QUIT:** Inviato da un client durante un match attivo per abbandonare la partita in corso, decretando la vittoria automatica dell'avversario per abbandono.
- **DISCONNECT:** Messaggio di controllo critico inviato dal client per segnalare la volontà di chiudere la sessione di rete o di una disconnessione improvvisa.
- **REMATCH_ACCEPT:** Inviato in risposta a una proposta di rivincita al termine di un match, per resettare la scacchiera e avviare immediatamente un nuovo incontro con lo stesso avversario.
- **REMATCH_DECLINE:** Inviato per rifiutare la proposta di rivincita, chiudendo definitivamente la sessione della stanza e riportando i giocatori nella lobby principale.

Avendo scelto il protocollo TCP che è orientato al flusso continuo di byte, per evitare la frammentazione o il mescolamento dei messaggi si impone al client e al server di utilizzare il carattere `\n` come delimitatore di ogni messaggio. L'invio dei messaggi viene centralizzato in una singola funzione (sia nel lato server, sia nel lato client) così da facilitare questa logica.

Capitolo 2

PROGETTAZIONE DEL SERVER CONCORRENTE IN C

2.1 Parallelismo

Per gestire la connessione simultanea di più utenti e consentire lo svolgimento in parallelo di più partite, l'architettura del server si basa sull'utilizzo dei Thread POSIX rispetto alla duplicazione dei processi tramite `fork()`. Questa scelta presenta precise motivazioni di efficienza e semplicità strutturale:

- **Condivisione dello Spazio di Indirizzamento:** A differenza dei processi, che possiedono uno spazio di memoria isolato e richiederebbero complessi canali di comunicazione interprocesso (IPC) come le pipe per scambiarsi dati, tutti i thread di un processo condividono lo stesso segmento di memoria (segmento dati e heap). Di conseguenza, i thread dei singoli client possono leggere e modificare direttamente le strutture dati globali, eliminando la necessità di creare, gestire e sincronizzare strutture di comunicazione intermedie.
- **Efficienza delle Risorse di Sistema:** La creazione di un thread e il passaggio di esecuzione (context switch) richiedono un consumo di memoria e CPU notevolmente inferiore rispetto a un processo intero.

2.2 Gestione Dinamica delle Risorse

Il thread principale del server esegue un ciclo infinito regolato dalla chiamata di sistema bloccante `accept`. Il suo unico compito è intercettare le richieste di connessione in ingresso e delegarne immediatamente la gestione a thread indipendenti tramite la funzione `pthread_create`.

Al fine di gestire l'intero ciclo di vita di queste connessioni senza gravare sul thread principale, l'architettura sfrutta la funzione `pthread_detach`. Questa scelta progettuale evita il ricorso a chiamate bloccanti come `pthread_join` (che congelerebbero il thread principale in attesa della terminazione dei client) e delega direttamente al sistema operativo il compito di deallocare e ripulire automaticamente tutte le risorse di un thread non appena il giocatore si disconnette.

Infine, la stabilità del sistema è presidiata da una politica di controllo preventivo basata sulla soglia massima di connessioni simultanee, definita dalla costante `MAX_CLIENT` (impostata a 100).

2.3 Sincronizzazione e Mutua Esclusione tramite Mutex

Essendo l'architettura multi-thread (e condividendo di conseguenza lo stesso spazio di indirizzamento), l'accesso concorrente alle strutture dati globali da parte di più thread contemporaneamente rappresenta una sezione critica altamente vulnerabile. Se due thread tentassero di modificare simultaneamente lo stato di una partita o l'elenco dei client, si verificherebbero inevitabilmente corruzioni di memoria e comportamenti imprevisti (Race Conditions).

Per prevenire queste anomalie, il design del server si affida a un rigoroso sistema di locking basato su variabili di mutua esclusione (`pthread_mutex_t`). La logica di sincronizzazione è applicata in modo granulare per non creare colli di bottiglia:

- **Protezione dell'Array dei Client:** L'inserimento o la rimozione di un socket descriptor dall'array globale delle connessioni attive è protetto dal `mutex_clients`. Prima che il thread principale (o un thread in disconnessione) possa cercare un indice libero o liberare uno slot, deve acquisire il lock, eseguendo l'operazione in totale isolamento per poi rilasciarlo immediatamente.
- **Protezione dello Stato di Gioco:** Anche le strutture che modellano le stanze di gioco sono sottoposte a mutua esclusione. Quando due client inviano comandi simultanei (es. un tentativo di mossa contemporaneo), il mutex della stanza assicura che il server valuti il turno, aggiorni la griglia di Forza 4 e verifichi l'eventuale condizione di vittoria in maniera atomica, impedendo sovrascritture incrociate dei dati.

2.4 Gestione delle Disconnessioni

L'architettura del server adotta una strategia di gestione unificata e simmetrica per la terminazione delle sessioni, trattando allo stesso modo sia la disconnessione volontaria (notificata tramite il comando applicativo `DISCONNECT`) sia l'interruzione improvvisa dovuta a guasti di rete o crash del client.

Una volta rilevata la disconnessione, il server avvia una procedura di ispezione dello stato del client per determinare la corretta azione di cleanup: viene verificato se l'utente si trovava semplicemente in attesa nella lobby o se era impegnato in una sessione di gioco attiva.

In quest'ultimo scenario, per garantire la coerenza distribuita dello stato del gioco ed evitare che l'altro client rimanga indefinitamente bloccato in attesa di una mossa, il server interviene tempestivamente notificando l'avversario del guasto di rete occorso, chiudendo la partita in corso e assegnandogli la vittoria d'ufficio per abbandono. L'intera rimozione del client dall'array globale dei descrittori attivi viene infine eseguita sotto mutua esclusione (come descritto nel paragrafo precedente) per prevenire corruzioni di memoria concorrenti prima della chiusura definitiva del socket.

2.5 Politiche di Gestione delle Stanze e Stati della Partita

Come richiesto esplicitamente dalle specifiche del progetto, il server deve monitorare e gestire in modo coerente il ciclo di vita di ogni singola stanza di gioco. La transizione tra gli stati della partita è stata implementata sul backend per garantire che la logica di business sia centralizzata e non manipolabile dal client.

Ogni partita creata segue un diagramma a stati rigido:

- **Stato Iniziale / Nuova Creazione:** Quando un utente avvia una sessione, la partita viene registrata nel sistema con un identificativo univoco (`id_match`), assumendo lo stato di attesa. In questa fase, il server aggiorna la lobby globale per informare tutti gli altri client connessi della disponibilità della stanza.
- **Fase di Negoziazione (Sfida):** Quando un secondo giocatore richiede l'accesso a una stanza specifica tramite il suo ID, il server blocca temporaneamente la stanza e interroga il creatore (host). L'host ha la facoltà di accettare o rifiutare la richiesta. In caso di rifiuto (`REJECT`), la stanza torna immediatamente disponibile nella lobby.
- **Stato di Gioco Attivo:** Se la sfida viene accettata (`ACCEPT`), la partita cambia stato. Da questo momento, il server isola la comunicazione tra i due sfidanti, inviando i dati di aggiornamento della griglia solo a loro. Gli altri utenti della lobby ricevono una notifica di stato che impedisce qualsiasi ulteriore tentativo di join.
- **Stato Terminata ed Esito Personalizzato:** Al raggiungimento di una condizione di vittoria o pareggio, il server valuta la griglia, dichiara la fine del match e distribuisce un esito personalizzato e asimmetrico ai due giocatori (Vittoria/Sconfitta), mentre invia una notifica generica di conclusione a tutti i client.

Capitolo 3

PROGETTAZIONE DEL CLIENT IN PYTHON

3.1 User Interface e Threading asincrono

Il client (frontend) dell'applicazione è sviluppato in Python utilizzando la libreria grafica Tkinter. L'architettura del client è progettata per essere totalmente asincrona e non-bloccante rispetto alle operazioni di rete.

Il problema principale nella progettazione di client con GUI è il congelamento dell'interfaccia: se il thread principale di Tkinter (il *Main Event-Loop*, responsabile del disegno della finestra e della reazione ai click dell'utente) effettuasse direttamente chiamate di rete bloccanti come `socket.recv()`, l'intera applicazione si "congelerebbe" in attesa di una risposta dal server, risultando inutilizzabile.

Si è adottato quindi un approccio multi-thread:

- **Thread della GUI (Main Thread):** Gestisce esclusivamente il rendering visivo della griglia, la navigazione nei menù e cattura l'input dell'utente.
- **Thread di Rete (Daemon Thread):** Avviato in background tramite il modulo `threading`, il metodo `listen_server` del `NetworkController` resta costantemente in ascolto sul socket TCP, intercettando i messaggi in ingresso inviati dal server in C.

3.2 Sincronizzazione Thread-Safe

Poiché le librerie grafiche come Tkinter non sono thread-safe (solo il Main-Thread può modificare la UI), il thread di rete non può invocare direttamente le funzioni che modificano lo schermo.

Per scambiare i messaggi tra i due thread in modo sicuro, si implementa il pattern Produttore-Consumatore. Quando il thread di rete riceve un comando dal server, utilizza meccanismi di sincronizzazione sicura (come la libreria `queue.Queue` di Python, che gestisce implicitamente i lock al suo interno, o l'utilizzo di callback sincronizzate nel loop di Tkinter tramite il metodo `.after()`) per notificare la GUI.

In questo modo, l'interfaccia grafica si aggiorna in tempo reale consumando gli eventi di rete, rimanendo sempre fluida e reattiva alle interazioni dell'utente locale.

Capitolo 4

ISOLAMENTO E ORCHESTRAZIONE DEL DEPLOYMENT

4.1 Containerizzazione dell'Infrastruttura: Server e Client

Lo sviluppo di applicazioni di rete in C è fortemente legato all'architettura e al sistema operativo su cui il codice viene compilato. Allo stesso modo, un client grafico in Python basato su librerie specifiche richiede ulteriori librerie di sistema per il rendering visivo che potrebbero non essere preinstallate o differire sulla macchina ospite.

Per evitare i classici problemi legati all'ambiente host (librerie di sistema mancanti, versioni differenti del compilatore GCC o dell'interprete Python ecc...), l'infrastruttura è stata containerizzata utilizzando Docker. Questo approccio garantisce un ambiente di esecuzione standardizzato, isolato ed immutabile:

- **Isolamento del Backend (C):** Garantisce che il server venga sempre compilato ed eseguito con gli header standard Linux (es. `pthread.h`, `sys/socket.h`), annullando i problemi di portabilità del codice nativo.
- **Isolamento del Frontend (Python/Tkinter):** L'ambiente dedicato al client si occupa di configurare in totale autonomia una versione ottimizzata di Python, installando automaticamente tutti i componenti di sistema necessari per la corretta visualizzazione dell'interfaccia grafica. Inoltre si è scelto di non avviare l'applicazione con i massimi privilegi di amministratore. Il container provvede invece a creare un utente standard (apprunner), dotato di permessi limitati e isolati, delegato esclusivamente all'esecuzione del gioco.

Questo garantisce una portabilità assoluta del progetto su qualsiasi macchina host. L'utente finale ha come prerequisito solamente Linux, Docker e Docker Compose.

4.2 Integrazione e Networking tramite Docker Compose

Per semplificare la fase di test e l'avvio dell'infrastruttura, si è scelto di utilizzare Docker Compose. Tramite un singolo file di configurazione `docker-compose.yml` l'intera applicazione viene orchestrata con un solo comando `docker compose up --build`.

Il Compose si occupa di definire le regole di networking e gestisce anche il deployment rapido del client grafico, passando le necessarie variabili d'ambiente (come la variabile `$DISPLAY` nei

sistemi Linux) per permettere al container di comunicare in sicurezza con il server grafico locale e renderizzare l'interfaccia di Tkinter direttamente sul monitor dell'utente.